Sr. No. 6174

Space For Marks | Question No. | START WRITING HERE (Begin answer for each question on a new page)

Q1 a] $f(w) = \frac{1}{2}\|Xw - y\|^2$

For gradient update rule, we know that

$$x_{t+1} = x_t - \eta \cdot f'(x)_{x=t} \quad \text{—— (1)}$$

Here $f'(x)$ is the slope at $x = t$.
But in the given eqn, instead of $f(x)$ we have a complex func^n $f(w)$.

$$\therefore f'(w) = \frac{d(f(w))}{dw}$$

$$= \frac{d}{dw}\left[\frac{1}{2}(Xw - y)^2\right]$$

$$= \frac{1}{2} \cdot 2(Xw - y)(X)$$

$$f'(w) = (Xw - y)\, X. \quad \text{—— (2)}$$

From the equation (1) we can write gradient update rule for gradient descent as

$$w_{t+1} = w_t - \eta f'(w)_{w=t} \quad \text{—— (3)}$$

Here, w_t = current value
w_{t+1} = next value
η = learning rate
$f'(w)$ = slope of function at $w = t$.

Substituting value from eqn (2) in eqn (3) we get

$$w_{t+1} = w_t - \eta \cdot [(Xw - y) \cdot X]$$

This is the required eqn for gradient update rule.
Now, writing this in matrix form we get,

slope $A = (Xw - y) \cdot X$
$= X^2 w - yX$.
hence matrix $A = [X^2 - yX]$

Space For Marks | Question No. | START WRITING HERE (Begin answer for each question on a new page)

Q19] Now, let z be the func^n to represent the gradient update rule

$$\therefore z = w_t - \eta \cdot A$$

But we know that,

$$w_t = w_{t-1} - \eta A$$
$$w_{t-1} = w_{t-2} - \eta A$$

and so on ... $w_1 = w_0 - \eta A$

$$\therefore z = w_0 - t \cdot (\eta A)$$

But Initially say w_0 is an identity matrix

$$\therefore w_0 = I$$

$$z = (I - \eta A) t$$

Now, to find eigenvalue we have

$$|I - \lambda \eta A| = 0$$

Also, $A = X^2 - yX$

$$|I - \lambda\eta \cdot (X^2 - yX)| = 0$$

Now for orthogonal matrix, $X^2 = X^T X$.

so on ~~des~~ solving the eq^n we get

$$2 \cdot \lambda\eta - 2 = 0 \qquad \lambda\eta = 0$$
$$\lambda\eta = 2 \qquad \boxed{\eta = 0}$$
$$\boxed{\eta = \frac{2}{\lambda}}$$

Hence from these 2 eq^n we can say that learning rate η must be less that $\frac{2}{\lambda_{max}}$ as it is obtained from eigenvalue which is maximum.

Sr. No. 6174

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q1b)	

Space For Marks | Question No. | START WRITING HERE (Begin answer for each question on a new page)

Q29] * For standard, 2D convolution the complexity will be as followed —

For a filter of f x f, there will be f x f no. of multiplications for one single window.

In a H x W sized image, the window will be moved certain no. of times which will give us the feature map.
So we can say that no of cells in feature map is equal to the no. of times filter is convoluted.

∴ size of feature Map ⇒

$$(H - f + 1) \times (W - f + 1)$$

So total complexity will be ⇒
size of feature map x size of filter

$$(H - f + 1) \times (W - f + 1) \times f \times f \quad \text{---} \star$$

* for depth wise seperable convolution the complexity would be increased as depth (channels) are added in the filter.

Hence for each level of depth, the 2D convolution would be performed hence total multiplication would be

$$(H - f + 1) \times (W - f + 1) \times f \times f \times C$$

* In modern architectures, they are important because it allows to network to learn more critical & minute features.

Sr. No. 6174

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q29]	* It gives the network, a deeper understanding of position, color, shadow depth, etc in an image.
		* All these parameters are really important in modern world applicaⁿ like object dectection, computer vision etc. especially on RGB scale images
		Hence depthwise seperable convolutions are critical in modern architectures.

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q2b] * A linear undercomplete autoencoder trained with MSE performs similar to PCA.

In PCA, we try to use those features which give max. variance. This is similar to what happens when we use MSE for undercomplete autoencoder.

It simply tries to reduce the MSE instead of learning important features required for regeneration.

* An overcomplete autoencoder has extra dimensions in latent / bottleneck than original input.

If not regularized it will simply learn the entire input & map identically in output via decoder.

No real compression, learning happens in such cases when regularization is not used. It simply does $f(x) = x$.

which is Identity function, hence it is said that overcomplete autoencoders ~~has~~ without regularization tends to learn identity function.

Sr. No. 6174

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Q3a] For a RNN,

$f(y) = \sigma(w_{t-1} \cdot h_{t-1} + w_i \cdot x_i + b_i)$

Now,

$L = f(h) = h_{t-1} \cdot f(y)$

$$\frac{\partial L}{\partial h_{t-k}} = \frac{\partial}{\partial h_{t-k}} [h_{t-1} \cdot f(y)]$$

$$\frac{\partial L}{\partial h_{t-1}} = \frac{\partial}{\partial h_{t-1}} h_{t-1} \cdot f(y)$$

But this depends upon $f(y)$ which again has h_{t-k} terms in it.

$$\frac{\partial L}{\partial h_{t-1}} = \frac{\partial}{\partial h_{t-1}} [w_{t-1} \cdot h_{t-1} + w_i x_i + b_i]$$

Recursively unrolling this RNN, we get

$$\frac{\partial L}{\partial h_{t-k}} = \frac{\partial (w_{t-1} \cdot h_{t-1})}{\partial h_{t-1}} \cdot \frac{\partial (w_{t-2} \cdot h_{t-2})}{\partial h_{t-2}} \cdots \cdots \frac{\partial (w_0 \cdot h_0)}{\partial h_0}$$

As there are multiple terms & the activn used is sigmoid, it will always give value ≤ less than 0.5. which causes vanishing Gradient problem.

The forget gate in LSTM modifies this & prevents a vanishing gradient problem by using Sigmoid Activn in a smart way.

In LSTM, forget gate eqn is

$$f_t = \sigma(w_f \cdot [h_{t-1}, x_t] + b_f)$$

Sr. No. 6174

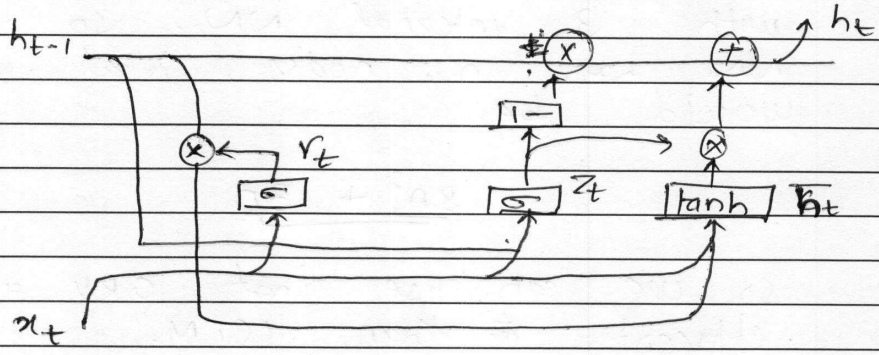
Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q3a]	

Sr. No. 6174

Space For Marks | Question No. | START WRITING HERE (Begin answer for each question on a new page)

Q3b] The gates in GRU are - reset gate and update gate.

Architecture



From this architecture we can see that Reset gate r_t ~~has~~ eqn & update gate z_t

$$r_t = \sigma(w_r \cdot [h_{t-1}, x_t] + b_r)$$

$$z_t = \sigma(w_z \cdot [h_{t-1}, x_t] + b_z)$$

from these gates we have potential current state $\tilde{h}_t$ and current state h_t

$$\tilde{h}_t = \tanh(w_c \cdot [r_t \cdot h_{t-1}, x_t] + b_c)$$

$$h_t = h_{t-1} \cdot (1 - z_t) + z_t \cdot \tilde{h}_t$$

These are the 4 required eqn of GRU.

In LSTM there are 3 gates, Forget Input & output which internally implement 5 neural networks each say of nodes n.

So for 5 NN with 'n' nodes, there will be $4n^2$ weights to be trained and $4n$ bias to be trained

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
	Q3b)	So for ~~RNN~~ LSTM total updates will be $4n^2 + 4n$
		While in GRU we have only 2 gates with 3 internal NN. So if each NN has 'n' nodes, total updates would be
		$3n^2 + 3n$.
		So we can se that GRU are much efficent ~~#~~ than LSTM.
		In real world for text predictions, POS, NER, this diff becomes very large as nodes may go up to 500 per NN.
		So diff $= n^2 + n$
		$= 500^2 + 500$
		$= 25 \times 10^4 + 500$ extra calculations for each epoch.

Sr. No. 6174

START WRITING HERE

(Begin answer for each question on a new page)

Space For Marks	Question No.	

Sr. No. 6174

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Sr. No. 6174

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)

Sr. No. 6174

Space For Marks	Question No.	START WRITING HERE (Begin answer for each question on a new page)
		ROUGH WORK